Hi! I am Ronit and I will be shedding light on the implementation mechanics of this paper.

To see how the system retrieves images, we follow a pipeline:
➢ First we tune images into useful vectors.
➢ Then we take input queries and retrieve their descriptor vectors.
➢ Then we organize them using hierarchical k-means into a neat tree.
➢ Then we let a query descriptor traverse through this tree to rank candidate images.
➢ And finally, we return the closest matching image to our query.
A query for all practical purposes, with respect to this paper is an input image or set of input images.

—NEXT-14

Our first step in this pipeline shall revolve around extracting high quality global descriptors of each image without losing any context or valuable information. Broadly, the system supports two retrieval regimes, and this determines everything else:

1. category-level retrieval, where we want images sharing the same semantic class, and
2. particular-object retrieval, where we want multiple views of the same physical instance.

—NEXT-15

For category-level tasks, the model uses a ViT/CLIP-based encoder called UNICOM. This encoder slices the image into 2D patches, embeds them using the patch embedding layer, **prepends** a learnable CLS token into this vector, pushes them through L transformer layers, and uses the final **learned** CLS token as a semantic global summary. Then it is linearly projected and normalized into a Δ-dimensional descriptor. (Δ is set by the authors - 256/512/1024)

—NEXT-16

For object-retrieval tasks, we use the R-GeM CNN pathway: the image passes through a convolutional backbone which produces Channels×H×W number of 2D feature maps, which is then aggregated against HxW using Generalized Mean Pooling so that spatial activations are collapsed into a single vector g_j. Then, same as before, linear projection, followed by L2 normalization giving us a Δ-dimensional descriptor. For high level retrieval, the number of channels is set to be high. The projection weights and GeM-exponent **p** aren't arbitrary at all. They're learned end-to-end through gradient updates during training.

—NEXT-17

Regardless of the encoder, every database image becomes a single global descriptor, and these descriptors form the set of DATABASE DESCRIPTORS whichever pathway we choose based on our data. For the query image d_Q, the exact same encoder is applied -

1. ViT/CLIP for category-level queries and
2. CNN+GeM for object-level queries

to produce a query descriptor v_Q living in the same feature space as the dataset vectors.

—NEXT-18

Now we enter the OFFLINE REGIME of our algorithm. Once all descriptors are computed, the system builds a HKM tree offline. The root node holds all descriptors. We select hyperparameters k also called branching factor and L which is also known as tree depth. At level 1, we run k-means yielding k centroids and k buckets. Each bucket becomes a child node with its assigned descriptors. At level 2, each child node runs k-means again on its local subset, creating k more children, and this recursive clustering continues until depth L is reached.

Internal nodes store only their centroid vectors, while leaf nodes at depth L store the actual image descriptors assigned to them. The centroid vectors are separate from the original descriptor superset and act as a representative of its constituent descriptors.

—NEXT-19

Now we enter the ONLINE REGIME of our algorithm. We take v_Q and start at the root. For each internal node, we compute the similarity or distance between v_Q and all k centroids at that node. Then we pick the closest one, and descend into that child. This process, called intensive search (which will be later explained by Muhamed), follows a single best path down the tree from level 0 to level L.

—NEXT-20

When we reach a leaf node n, we collect all descriptors inside it—typically a few hundred instead of the full dataset. THIS IS THE BIGGEST JUSTIFICATION FOR USING HKM in the first place. HKM structures the descriptors into a multi-layered tree, letting the query narrow down the search space, so that searching behaves like a logarithmic walk instead of exploding with dataset size.

—NEXT-21

Now, for each candidate image descriptor in this leaf, we compute its similarity with the query, producing a similarity vector for each query image (singular or plural queries doesn't matter). Sorting this vector/s gives us the final ranking permutation π. The top-1 retrieved image is OUR REQUIRED ANSWER and thus the **end of implementation mechanics**.

—NEXT-22

I'll shortly go over performance evaluation metric before handing it off to Muhamed:

1. category-level retrieval uses Recall@1, which checks whether the class label of the top-ranked image matches the query label, while
2. particular-object retrieval uses mAP, which evaluates percentage precision across the entire ranked list because object-level queries often have many relevant instances.

THANK YOU!

—NEXT-23